

Comparative Analysis of Variable Ordering Heuristics in Backtracking-Based Sudoku Solving

Alfie Jenkins
School of Computing
Falmouth University
Cornwall, United Kingdom
aj313798@falmouth.ac.uk

Abstract—This paper compares three variable ordering heuristics for backtracking-based Sudoku solving across 200 standard 9×9 puzzles. MRV achieved zero backtracks on every puzzle; sequential selection averaged 95.70 (median 60) and random averaged 33,216 (median 14,517). Despite MRV’s higher per-iteration cost, it solves puzzles faster end-to-end because it avoids the exponential search branches that consume most of random selection’s time. Mann-Whitney U tests confirm all pairwise differences are highly significant ($p < 0.001$). Variable ordering is the dominant factor in backtracking solver performance.

Index Terms—constraint satisfaction, backtracking, heuristics, Sudoku, variable ordering, MRV

I. Introduction

Variable ordering determines how much work a backtracking solver actually does. Pick the right cell first and you may never backtrack at all. Pick the wrong one and the tree grows exponentially. This paper tests how large that difference is in practice.

Sudoku is a useful benchmark for this. The constraints are fixed, solutions are unique, and there are 6.67×10^{21} possible configurations — enough room for heuristics to show real differences.

We ran row-major selection, uniform random, and Minimum Remaining Values (MRV) across 200 puzzles, recording backtrack counts and solve times. The gaps turned out to be substantial — and the same logic applies to any real CSP solver. Scheduling systems, configuration tools, and resource allocators all pay for a poor heuristic on every run.

II. Background and Related Work

A. Constraint Satisfaction Problems

A CSP consists of three components: a set of variables $V = \{V_1, V_2, \dots, V_n\}$, domains $D = \{D_1, D_2, \dots, D_n\}$ specifying possible values for each variable, and constraints C that restrict valid value combinations [1]. Finding a solution requires assigning values to all variables such that all constraints are satisfied.

CSP solving is NP-complete in general [2]; backtracking with constraint propagation is the standard approach for exact solutions because exhaustive search is computationally infeasible.

B. Sudoku as a CSP

Sudoku can be formalized as a CSP with 81 variables (cells), each with domain $\{1, 2, \dots, 9\}$, and 27 all-different constraints (9 rows, 9 columns, 9 boxes) [3]. The problem is to assign values satisfying all constraints.

While general $n \times n$ Sudoku is NP-complete [2], standard 9×9 puzzles have finite complexity and can be solved efficiently with appropriate algorithms.

C. Backtracking Search

Backtracking explores the search space by recursively assigning values and undoing them when a constraint is violated [1]. Efficiency depends on minimising failed branches through good variable and value ordering.

D. Variable Ordering Heuristics

The Minimum Remaining Values (MRV) heuristic, also called the “most constrained variable” or “fail-first” heuristic, selects the variable with the fewest legal values remaining [4]. By choosing highly constrained variables first, MRV detects failures early and prunes the search tree aggressively.

Norvig demonstrated MRV’s effectiveness in Sudoku solving, integrating it with constraint propagation [4]. Russell and Norvig note that MRV consistently reduces search tree size across CSP domains by prioritising variables most likely to cause early failure [1].

When multiple cells share the minimum remaining value count, a tie-breaking strategy is required. The degree heuristic selects the variable involved in the most constraints with unassigned neighbours [1]. Our implementation breaks ties arbitrarily; degree-based tie-breaking is left as a future extension.

Value ordering — the sequence in which candidate values are tried for a given cell — is a complementary dimension to variable ordering. Russell and Norvig describe the Least Constraining Value (LCV) heuristic, which prefers values that rule out the fewest options for neighbouring variables [1]. Our work focuses on variable ordering, leaving value ordering as a natural extension.

E. Constraint Propagation

Constraint propagation techniques reduce variable domains during search, pruning the search space before backtracking occurs. Forward checking removes from neighbouring domains any values that conflict with a new assignment [8]. Arc consistency (AC-3) extends this by enforcing constraints between all pairs of variables, ensuring that for every value in one variable’s domain there exists a compatible value in each neighbour’s domain [7].

Norvig’s Sudoku solver combines MRV with constraint propagation, achieving greater efficiency than backtracking alone by eliminating infeasible values before they are explored [4]. Propagation is deliberately excluded. The experiment isolates variable ordering; combining it with propagation would make it impossible to attribute performance differences to heuristic choice alone.

III. Methodology

A. Implementation

We implemented a standard recursive backtracking solver in Python 3.14 [1]. It works as follows:

```

find next empty cell using heuristic
if no empty cells then return solution found end if
for each value  $v \in \{1 \dots 9\}$  do
    if  $v$  is valid at cell then
        assign  $v$  to cell
        if recursive solve succeeds then return success end if
        remove  $v$  from cell; increment backtrack counter
    end if
end for
return failure

```

A value is valid if it does not already appear in the same row, column, or 3×3 box.

B. Variable Ordering Heuristics

Each strategy changes only one thing: which empty cell to assign next.

Sequential: Cells are visited left-to-right, top-to-bottom. No constraint information is used.

Random: A cell is chosen uniformly at random from all empty cells. This tests whether any ordering at all beats chance.

MRV: For each empty cell, count how many values remain valid and pick the most constrained one. Ties break arbitrarily. The reasoning: a cell with one valid value must take it; a cell with none has already failed. Finding that failure early avoids wasted assignments downstream.

C. Experimental Design

Dataset: The first 200 puzzles from the Kaggle Sudoku dataset [5] were used. Puzzles are represented as 81-character strings with '0' for empty cells. The dataset is released under a CC0 public domain licence, permitting

unrestricted academic use; no personally identifiable data is present.

Procedure: A fixed random seed (42) was set before all runs to ensure reproducibility of the random heuristic. Each puzzle was solved once with each heuristic. The board state was reset between runs to ensure identical starting conditions. For each run, we measured:

- Execution time (seconds)
- Number of backtracks
- Solution correctness (verified against provided solutions)

Platform: Experiments ran on Arch Linux with Python 3.14, pandas 2.0, and numpy 1.24. Code is available at <https://github.com/AlfieMoon21/sudoku-heuristics>.

IV. Results

Tables I and II present performance and statistical test results. Figure 1 shows average backtracks per heuristic.

TABLE I
Performance Summary Statistics (n=200)

Heuristic	Mean BT	Med. BT	Time (s)	Max BT
Sequential	95.70	60	0.000421	640
Random	33,216	14,517	0.185786	582,024
MRV	0	0	0.004142	0

TABLE II
Mann-Whitney U Test Results (two-sided)

Comparison	U statistic	p-value
Sequential vs Random	267	2.58×10^{-65}
Sequential vs MRV	40,000	2.35×10^{-76}
Random vs MRV	40,000	2.36×10^{-76}

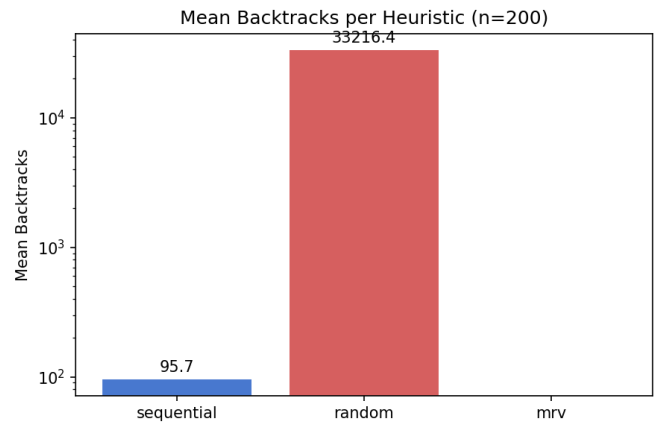


Fig. 1. Average backtracks per heuristic

MRV achieved zero backtracks on all 200 puzzles. This result should be interpreted in the context of dataset difficulty: all puzzles are standard, well-formed 9×9 grids with unique solutions. Harder instances — or adversarially

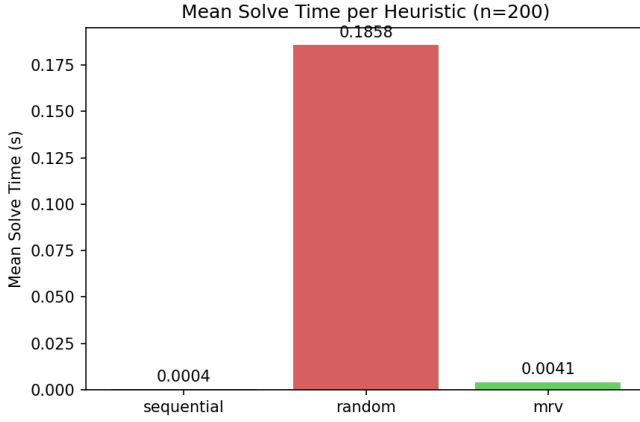


Fig. 2. Average solving time per heuristic

constructed puzzles with minimal givens — may still require backtracking under pure MRV without constraint propagation [2].

Sequential selection averaged 95.70 backtracks (median 60, $\sigma=103.69$). The gap between mean and median indicates a right-skewed distribution: many puzzles are solved with few backtracks, while a tail of structurally challenging puzzles inflates the mean. Figure 3 shows this distribution.

Random selection performed worst, averaging 33,216 backtracks (median 14,517, $\sigma=54,529$) with maximum 582,024. The 347-fold mean increase over sequential (242-fold by median) confirms that variable ordering determines solver efficiency more than any other factor tested.

Despite solving puzzles without backtracking, MRV’s mean execution time (0.00414s) was $9.8\times$ slower than sequential (0.000421s), reflecting the per-iteration overhead of computing remaining values for all empty cells.

However, random selection’s 0.186s average shows that backtracking cost dominates for uninformed heuristics. The $440\times$ solve-time gap between random and MRV has direct energy consequences at scale: a scheduling system calling the solver thousands of times a day burns that overhead on every run.

A. Statistical Significance

Mann-Whitney U tests (Table II) confirm that all pairwise differences are highly significant ($p < 0.001$). A non-parametric test was chosen because backtrack distributions are strongly right-skewed. The U statistic of 40,000 for both Sequential vs MRV and Random vs MRV equals $n_1 \times n_2 = 200 \times 200$, the theoretical maximum, confirming that every MRV run produced strictly fewer backtracks than any Sequential or Random run.

B. Puzzle Difficulty Correlation

Figure 4 plots empty cell count against sequential backtracks. Pearson correlation yielded $r = 0.043$ ($p = 0.546$). Empty cell count, the most natural proxy for puzzle difficulty, explains essentially none of the variance in

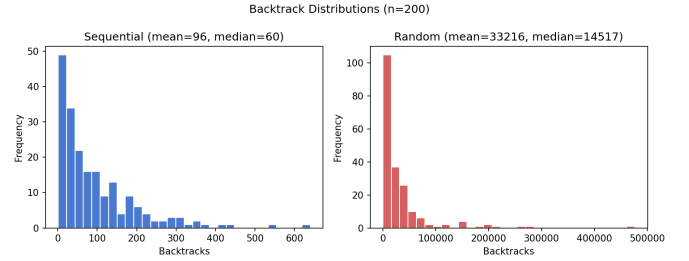


Fig. 3. Distribution of backtracks per heuristic across 200 puzzles. MRV omitted (all zero).

sequential backtracks. Performance appears to depend on the spatial arrangement of given cells rather than their count.

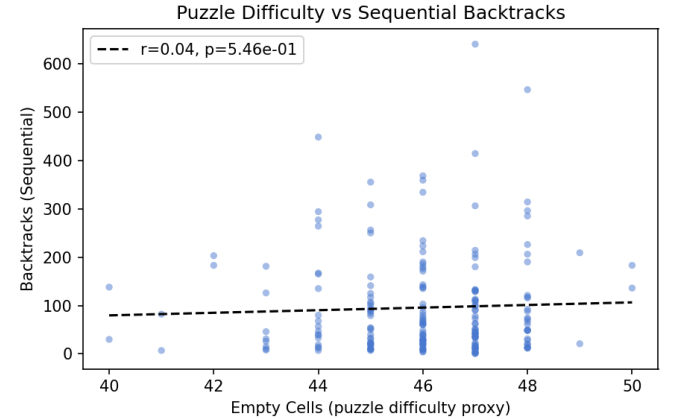


Fig. 4. Empty cell count vs sequential backtracks ($r = 0.043$, $p = 0.546$). No significant correlation.

V. Discussion

A. Why MRV Achieves Zero Backtracks

MRV’s performance follows from the fail-fast principle. By always selecting the most constrained cell, MRV either finds a cell with exactly one valid value and places it without risk, or finds a cell with no valid values and fails immediately rather than after wasted assignments. For the puzzles tested, this generates a chain of forced moves that resolves the board without any speculation.

B. Computational Trade-offs

MRV incurs $O(n \cdot k)$ overhead per cell selection, where n is empty cells and k is domain size (9). For a 9×9 Sudoku, this adds approximately 81 constraint checks per iteration.

Sequential ordering has $O(1)$ cell selection but explores exponentially more search branches. Our results show MRV’s linear overhead is vastly preferable to exponential search.

Russell and Norvig observe the same trade-off for CSPs generally: the per-step overhead of a well-chosen heuristic is almost always recovered through reduced search [1].

C. Implications for CSP Solving

The scale of these differences suggests the finding is not Sudoku-specific. A 347-fold backtrack increase from random ordering, and complete elimination under MRV, shows that heuristic choice is the dominant factor in solver performance, not a secondary design decision.

D. Spatial Arrangement and Puzzle Difficulty

The non-significant difficulty correlation ($r = 0.043$, $p = 0.546$) reveals a structural property of the sequential heuristic. Given-cell count is the most natural proxy for puzzle difficulty, yet it explains essentially none of the variance in sequential backtracks. This suggests that the sequential heuristic’s performance is governed by the topology of the constraint graph at solve time: whether early row-major cells happen to be highly constrained relative to their neighbours, rather than how many cells are empty overall. A puzzle with 51 empty cells arranged so that the first few empty cells are in sparse regions may require far more backtracking than a puzzle with 55 empty cells whose first empty cells are tightly constrained. This points toward structural puzzle metrics—constraint graph topology, neighbourhood density—as more meaningful difficulty predictors than cell count for sequential solving.

E. Limitations

Our study focuses on standard 9×9 Sudoku from a single dataset. All 200 puzzles are drawn from the same distribution of difficulty and construction method; results may differ for puzzles generated by different methods, for harder puzzles with fewer givens, or for larger grid variants. Additionally, we examined only three heuristics; hybrid approaches combining MRV with degree heuristics or value ordering strategies merit investigation.

VI. Conclusion

Across 200 standard Sudoku puzzles, MRV eliminated backtracking entirely while random ordering averaged 33,216 backtracks — a 347-fold increase over sequential. MRV pays $9.8\times$ more per cell selection but recovers that cost many times over: it solves puzzles $44\times$ faster than random by avoiding the search branches that consume most of random selection’s time. Mann-Whitney U tests confirm all differences are highly significant ($p < 0.001$); the U statistic for MRV comparisons reached the theoretical maximum of 40,000, meaning MRV outperformed both alternatives on every single puzzle.

A. Legal, social, ethical, professional, and sustainability considerations

Legal. CSP solvers deployed in safety-critical systems—air traffic scheduling, medical resource allocation—may face regulatory requirements on algorithmic auditability. When a poorly chosen heuristic causes solver timeouts in those contexts, the failure is not just technical. Documenting algorithm choices and known limitations is standard professional practice for exactly this reason.

Social. A suboptimal variable ordering does not merely slow a solver; it narrows the portion of the solution space explored. In workforce scheduling or exam timetabling, that means some arrangements are never found—and the ones that are found may be valid but unfair, consistently disadvantaging the same people. The 347-fold backtrack gap here illustrates the scale of difference that heuristic choice alone can introduce.

Ethical. MRV achieves zero backtracks on this dataset. That result is specific to standard, well-formed 9×9 grids. Harder or larger instances may still require backtracking, and a system that presents heuristic output as an optimal solution without qualification is misleading—regardless of how well the heuristic performs on benchmarks.

Professional. Sequential selection is straightforward to implement and audit. MRV is more complex and requires careful testing. Neither is obviously correct without understanding the trade-offs; choosing the wrong one for a production system where solver calls accumulate under load is a foreseeable risk. The ACM Code of Ethics [6] requires practitioners to understand the limitations of the techniques they deploy.

Sustainability. Random selection runs 440 times slower than MRV on average. In a scheduling system running thousands of solver invocations daily, that is not an abstract concern—it is energy consumption. Heuristic selection is usually framed as a correctness or speed decision. The data here suggest it should also be treated as a resource decision.

B. Future Work

Testing harder puzzle variants and other CSP domains would establish whether MRV’s perfect backtrack elimination is dataset-specific. Adding constraint propagation would show whether it eliminates the residual backtracking seen under sequential ordering, and at what additional overhead. The spatial arrangement result also suggests that structural puzzle metrics—constraint graph topology, neighbourhood density—may predict sequential solver difficulty better than cell counts do.

References

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- [2] T. Yato and T. Seta, “Complexity and completeness of finding another solution and its application to puzzles,” *IEICE Trans. Fundamentals*, vol. E86-A, no. 5, pp. 1052–1060, 2003.
- [3] H. Simonis, “Sudoku as a constraint problem,” in *CP Workshop on Modeling and Reformulating Constraint Satisfaction Problems*, vol. 12, pp. 13–27, 2005.
- [4] P. Norvig, “Solving every Sudoku puzzle,” 2006. [Online]. Available: <https://norvig.com/sudoku.html>
- [5] B. Park, “Sudoku dataset,” Kaggle, 2017. [Online]. Available: <https://www.kaggle.com/datasets/bryanpark/sudoku>
- [6] ACM, “ACM Code of Ethics and Professional Conduct,” 2018. [Online]. Available: <https://www.acm.org/code-of-ethics>
- [7] A.K. Mackworth, “Consistency in networks of relations,” *Artificial Intelligence*, vol. 8, no. 1, pp. 99–118, 1977.
- [8] R.M. Haralick and G.L. Elliott, “Increasing tree search efficiency for constraint satisfaction problems,” *Artificial Intelligence*, vol. 14, no. 3, pp. 263–313, 1980.